CS4475 Fall 2026
Herbert Lawson
hrlawson@miners.utep.edu

Lab Report
Submitted on 09/23/2026

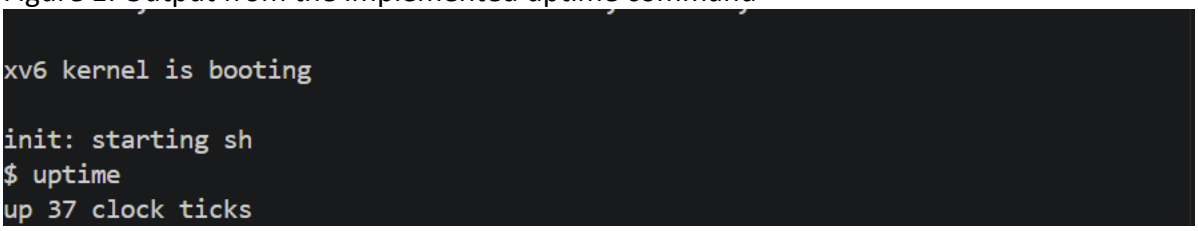
PA 1: Implementing xv6 User Commands and System Calls

Repo URL: https://github.com/juniorlaw/xv6-riscv/tree/pa1-syscalls

Task 1. (5 pts) Implement the uptime utility

I created uptime.c and added it to UPROGS in the Makefile. It uses uptime() to print how many clock ticks have passed since xv6 started. I also changed the number of CPUs from 3 to 1.

Figure 1. Output from the implemented uptime command



Task 2. (10 pts) Implement a time1 command that reports elapsed time.

I created time1.c to measure how long a command takes to run. It uses fork() and exec() to run the command and wait() to wait for it to finish. I used uptime() before and after to calculate the elapsed time.

Figure 2. Testing the time1 command with sleep 10 and with no command argument.



Task 3. (10 pts) Keep track of how much cputime a process has used.

- I modified kernel/proc.h, kernel/proc.c, and kernel/trap.c to keep track of the CPU time used by each process.
- In kernel/proc.h, I added a cputime field to struct proc so that each process can store its CPU usage in clock ticks.
- In kernel/proc.c, I initialized cputime to 0 inside allocproc() so that every newly allocated process begins with no accumulated CPU time.

- In kernel/trap.c, I modified the timer interrupt handling in usertrap() and kerneltrap() so that the current process's cputime is incremented when it uses a time slice.
- These changes allow xv6 to maintain CPU usage information for each process, which is later returned through the new wait2() system call.

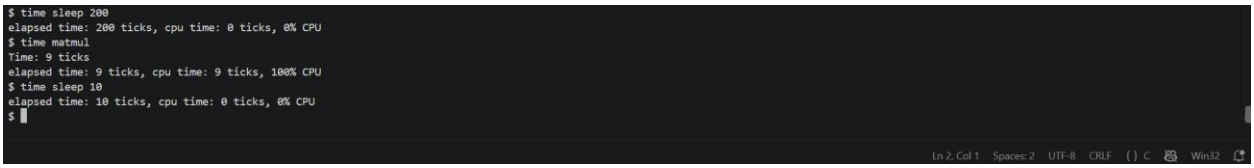
Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

- I implemented the new wait2() system call to wait for a child process to finish while also returning the child's CPU usage information.
- kernel/pstat.h - I created the rusage structure containing the cputime field.
- user/user.h - I added the declaration for struct rusage and the wait2() function prototype so user programs can call the new system call.
- user/usys.pl - I added an entry for wait2 so the user-level system call stub is generated.
- kernel/syscall.h - I assigned a system call number to wait2.
- kernel/syscall.c - I added sys_wait2 to the system call table so xv6 can route a user wait2() call to its kernel implementation.
- kernel/sysproc.c - I implemented sys_wait2(), which retrieves the user arguments and calls the kernel's kwait2() function.
- kernel/defs.h - I added the prototype for kwait2() so it can be called outside of proc.c.
- kernel/proc.c - I implemented kwait2() based on the existing kwait() logic. In addition to returning the child's exit status, it copies the child's cputime into a rusage structure and returns it to the parent process.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

I created time.c to show elapsed time, CPU time, and CPU percentage. It runs the command in a child process and uses wait2() to get its CPU time. I tested it with sleep and matmul to compare a waiting process with a CPU-heavy process.

Figure 3. Testing the time command with an I/O/wait-oriented sleep process and a CPU-intensive matmul process. sleep 10 consumed 0% CPU during the measured interval, while matmul consumed 100% CPU.



Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment helped me understand how system calls actually work in xv6. I also learned the difference between elapsed time and CPU time. The hardest part was setting up wait2() because it required changes across several files. I had an issue with the syscall mapping at first, but I found the incorrect entry and fixed it. I also had some problems building xv6 from Windows, so I moved it into WSL and got it working there.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One limitation is that our time command only tracks the direct child. If that command creates more child processes, their CPU time may not be included in the result. The timing is also measured in clock ticks, so very short programs may not be measured very precisely.